



clockplot: An R Package to Plot Event Times on a 24-Hour Clock

Abdullah Al Mahmud 
Bangladesh Cadet College

Abstract

Visualization is a fundamental component of statistical analysis, enabling clear data description and informed model selection. While various methods exist for visualizing event data, there is a notable gap in representing events on a clock face. This paper introduces **clockplot**, an R package for plotting timestamped events on a 24-hour circular clock face. This approach offers a powerful way to perceive the precise timing of events and facilitates intuitive comparisons with other events. The package is particularly useful for analyzing daily patterns, event clustering, and temporal gaps. For event times, it is significantly more revealing than conventional visualizations like bar or pie charts. The package also generalizes the clock plot approach to create cyclic charts for other time frames, including weekly and monthly cycles. This functionality enables effective event planning and pattern analysis across multiple periods, providing a new and insightful tool for temporal data analysis.

Keywords: circular visualization, temporal data, event times, clock plot, polar coordinates, R.

1. Introduction: Visualizing event times on a clock face

Visualizing temporal patterns in timestamped event data presents unique challenges, particularly when events exhibit cyclical behavior over 24-hour periods. Traditional linear timelines (e.g., using the **ggplot2** function `geom_point()` with time on the x-axis) have several limitations for this type of analysis:

1. **Circular continuity is broken:** The natural continuity between 23:59 and 00:00 is not preserved, making it difficult to see patterns that span midnight.
2. **Density patterns are hard to discern:** Clustering of events around specific times of day is less visually apparent on linear axes.

Feature	clockplot	circular	plotrix	ggplot2 + coord_polar()
Native timestamp input (HH:MM:SS)	✓	×	×	×
24-hour clock face	✓	✓	✓	Manual
Multivariate encoding (color + size + length)	✓	Limited	Limited	Manual
Clock hand length encoding	✓	×	×	×
Cyclic charts (day/week/month/year)	✓	×	×	Manual
Period planners	✓	×	×	×
ggplot2 compatible output	✓	×	×	✓
CRAN availability	✓	✓	✓	✓

Table 1: Comparison of **clockplot** with alternative packages for circular time visualization.

3. **Multiple days of data become cluttered:** Plotting several days of data on a linear timeline creates overlapping points that obscure daily patterns.

While several R and Python packages offer circular or radial visualization capabilities, they are either too general-purpose or not specifically designed for timestamp data:

- **General circular plotting:** Packages like **circular** (Agostinelli and Lund 2025) and **plotrix** (Lemon 2006) provide basic circular plotting, but require extensive data transformation for timestamp visualization.
- **Polar coordinates:** **ggplot2** (Wickham 2016) with `coord_polar()` can create circular plots, but requires manual conversion of timestamps to radians and lacks specialized aesthetics for event data.
- **Specialized but limited:** Packages like **chron** and **lubridate** (Grolemund and Wickham 2011) handle time data but don't provide circular visualization methods.
- **CircadiPy:** Deals with rhythmic data but does not create circular visualizations (Carvalho-Moreira, de Oliveira Guarnieri, Passos, Emrich, Bargi-Souza, Peliciari-Garcia, and Moraes 2024).

The **clockplot** package fills this gap by providing specialized visualization methods for timestamped event data on a 24-hour clock face. The package provides multivariate visualization capabilities including **colored point aesthetics** (mapping variables to hue/size/shape) and **modifiable clock hand lengths** (representing magnitude or intensity at each timestamp). These features enable clear differentiation of multiple variables on the same 24-hour clock face while maintaining intuitive interpretation of temporal patterns.

2. Available Software

Table 1 compares **clockplot** with existing alternatives for circular time visualization.

3. Acceptable Data Types

clockplot handles two primary data types:

3.1. Timestamped Event Data

For analyzing patterns in event occurrences, such as website visits, system logs, or biological rhythms:

```
# Sample univariate timestamps
time <- c("21:12:27", "02:20:09", "16:46:37", "08:57:52",
          "03:16:57", "03:35:33", "21:44:36", "12:45:15")
df <- data.frame(time = time)
clock_chart(df, time)
```

3.2. Multivariate Time Series Data

For visualizing continuous measurements at specific times, such as environmental monitoring or physiological data:

```
# Sample multivariate timestamps
df <- data.frame(
  time = c("21:12:27", "02:20:09", "03:16:57", "03:35:33"),
  temperature = c(20.1, 20.2, 21.4, 20.6),
  humidity = c(65, 70, 68, 72)
)
clock_chart_qnt(df, time = time, len = humidity, Col = temperature)
```

The package accepts flexible time formats: HH:MM:SS, HH:MM, or even H:M (e.g., 9:3). Seconds are parsed but have negligible visual impact.

4. Implementation

The construction of clock charts involves two main steps: (i) building the 24-hour clock skeleton, and (ii) plotting event times on the clock.

4.1. Clock Skeleton Construction

The clock skeleton is a unit circle with 24 hour marks and quarter-hour tick marks. In R, the circle is efficiently generated using complex numbers:

```
R> k <- 24
R> hour <- exp(1i * 2 * pi * (k:1) / k)
R> plot(hour, pch = 19)
```

For the full skeleton with quarter-hour marks:

```

R> k <- 24
R> subk <- 24 * 4
R> times <- exp(1i * 2 * pi * (k:1) / k)
R> subtimes <- data.frame(SubT = exp(1i * 2 * pi * (subk:1) / subk))
R> ampm <- c(rep(" AM", 6), rep(" PM", 12), rep(" AM", 6))
R> dfclock <- tibble(
+   time = times,
+   label = paste0(c(6:12, 1:5), ampm)
+ )
R> library(ggplot2)
R> p1 <- dfclock %>% ggplot() +
+   geom_path(data = subtimes, aes(Re(SubT), Im(SubT))) +
+   geom_line(data = subtimes %>% dplyr::slice(-c(2:95)), aes(Re(SubT), Im(SubT))) +
+   theme(aspect.ratio = 1) +
+   geom_text(data = dfclock, aes(Re(time) * 1.1, Im(time) * 1.1, label = label)) +
+   geom_point(data = subtimes, aes(Re(SubT), Im(SubT)), shape = 19, color = "black", size = 1.8)
+   geom_point(aes(Re(time), Im(time)), color = "black", size = 1.8)
R> p1 + theme(axis.text = element_blank(), axis.ticks = element_blank()) + labs(x = "", y = "")

```

This produces the static clock face with 24 hour labels (6 AM to 5 AM) and quarter-hour tick marks.

4.2. Coordinate Conversion Algorithm

To plot event times on the clock, timestamps in HH:MM:SS format are converted to Cartesian coordinates (x, y) on the unit circle:

1. Parse to fractional hours since midnight:

$$t_{frac} = hour + \frac{minute}{60} \quad (1)$$

Seconds are ignored as they contribute $< 0.1^\circ$ angular change.

2. Map to radians (clockwise from 12 o'clock = $\pi/2$):

$$\theta = \begin{cases} (6 - t_{frac}) \cdot \pi/12, & 0 \leq t_{frac} \leq 6 \\ (30 - t_{frac}) \cdot \pi/12, & 6 < t_{frac} \leq 24 \end{cases} \quad (2)$$

This piecewise formula handles the discontinuity at 6 AM/6 PM where the clock direction reverses relative to trigonometric angles.

3. Convert to Cartesian coordinates:

$$x = r \cdot \cos(\theta) \quad (3)$$

$$y = r \cdot \sin(\theta) \quad (4)$$

where $r = 0.95$ for event points (leaving room for labels at radius 1.1) and $r = 1.0$ for the clock skeleton.

Function	<code>geom_segment()</code>	<code>geom_point()</code>	Color
<code>clock_chart()</code>	Fixed	Fixed	Col argument
<code>clock_chart_col()</code>	Mapped to <code>crit</code>	Mapped to <code>crit</code>	Gradient <code>high/low</code>
<code>clock_chart_qnt()</code>	Mapped to <code>len</code> (color)	Mapped to <code>len</code> (color), <code>Col</code> (size)	Gradient <code>high/low</code>
<code>clock_chart_qlt()</code>	Mapped to <code>crit</code> (factor)	Mapped to <code>crit</code> (factor)	Discrete (Brewer)

Table 2: Aesthetic mappings for clock chart functions.

This algorithm is implemented in the internal functions `conv_data()`, `conv_data_col()`, and `conv_data_len()` (R/`conv_data.R`).

4.3. Mapping to **ggplot2** Aesthetics

Once coordinates (x_0, y_0, x_1, y_1) are computed (where $(x_0, y_0) = (0, 0)$ and (x_1, y_1) is the tip of the clock hand), the chart functions map these to **ggplot2** geoms:

All functions return a ‘**ggplot**’ object, enabling further customization with **ggplot2** functions (`labs()`, `theme()`, `facet_wrap()`, etc.).

4.4. Cyclic and Period Charts: Polar Bar Geometry

The cyclic charts (`cyclic_chart()`, `day_chart()`, `week_chart()`, `year_chart()`) and planners (`plan_day()`, `plan_week()`) use a fundamentally different approach: **polar bar charts** via **ggplot2**’s `coord_polar()` rather than clock hands.

- `cyclic_chart()`: General-purpose; maps any periodic variable to angle, values to radius via `geom_col()`.
- `day_chart()`: 24-hour rose plot; fixed hour labels; gradient fill.
- `week_chart()`: 7-day rose plot; fixed day labels (Saturday–Friday); gradient fill.
- `year_chart()`: 12-month rose plot; `month.name` labels; gradient fill.
- `plan_day()`/`plan_week()`: Categorical planners; `geom_col()` with `fill = activity` + `geom_text()` for labels; uses `scale_fill_brewer("Set2")`.

4.5. Design Decisions

5. Illustrations

We demonstrate **clockplot** using built-in datasets and a real-world case study.

5.1. Built-in Datasets

The package includes several datasets:

Decision	Rationale
24-hour clock (not 12-hour)	Avoids AM/PM ambiguity; matches HH:MM:SS format
Ignore seconds	Visual angle change $< 0.1^\circ$; adds noise without insight
Complex numbers for skeleton	<code>exp(1i * theta)</code> elegantly generates circle points
Hand length $\in [0.5, 0.95]$	0.5 avoids origin clutter; 0.95 leaves room for labels
Single hand (hour + minute fraction)	Cleaner for dense events; minute encoded as fractional position
<code>coord_polar()</code> for cyclic charts	Standard ggplot2 pattern; leverages existing polar system
Factor warning at > 5 categories	Human color discrimination limit; suggests <code>clock_chart()</code> instead

Table 3: Key design decisions and rationale.

- `smsclock`: SMS receiving times with sender/title metadata (32 events)
- `bdquake`: Earthquakes in Bangladesh since 2023 with magnitude, depth, time (54 events)
- `bdtemp`: Monthly temperatures for three Bangladeshi cities
- `brintcity`: Bus departure times from Dhaka
- `chatdf`: Chat message timestamps
- `gitcommit`: Git commit timestamps

5.2. Basic Clock Chart

```
R> data(smsclock)
R> p1 <- clock_chart(smsclock, time, Col = "green")
R> p1 + labs(title = "SMS Receiving Times")
```

The chart shows when messages are received. The largest gap is during night time (approximately 11 PM to 6 AM). Messages are received round the clock with few intermissions.

5.3. Colored Clock Chart

```
R> data(bdquake)
R> p <- clock_chart_col(bdquake, time = hms, crit = mag)
R> p + labs(size = "Magnitude", title = "Earthquakes in Bangladesh since 2023")
```

Earthquakes are colored by magnitude (green = low, red = high).

5.4. Chart with Modified and Colored Hands

```
R> p1 <- clock_chart_qnt(
+   data = bdquake, time = hms, len = depth,
+   Col = mag, high = "red", low = "blue"
+ )
R> p1 + labs(
+   color = "Depth", size = "Magnitude",
+   title = "Earthquakes in Bangladesh since 2023"
+ )
```

Point size represents magnitude, while color and hand length represent depth. The plot shows quakes are roughly randomly distributed with respect to time. Most earthquakes occurred between 1 PM to 5 PM. Two of the biggest gaps are between 5 PM to 9 PM (4 hours) and 10 PM to 2 AM (4.25 hours).

5.5. Qualitative Variable

```
R> clock_chart_qlt(smsclock, time = time, crit = Title) +
+   labs(color = "Sender", title = "SMS Received throughout the Day")
```

This shows which types of messages were sent when. Certain types (e.g., Supper_Offer) are always sent around the same time (4:30 PM to 6 PM).

5.6. Cyclic Charts

```
R> Col <- c("#0040ff", "#00bfff", "#8000ff")
R> cyclic_chart(bdtemp,
+   Period = Month, Value = Temperature,
+   crit = City, ColV = Col
+ )
```

Monthly temperatures for three cities.

```
R> value <- sample(15:30, 24, replace = TRUE)
R> day_chart(hvalue = value, high = "blue", low = "yellow", width = 0.8)
```

24-hour rose plot with gradient fill.

```
R> set.seed(10)
R> wtemp <- sample(10:40, 7)
R> week_chart(wtemp, high = "yellow") + labs(title = "Random Values by Day")
```

7-day rose plot.

```
R> syltmp <- c(18.4, 20.8, 24.3, 26.0, 26.8, 27.6, 28.0,
+             28.2, 27.9, 26.7, 23.3, 19.7)
R> year_chart(mvalue = syltmp)
```

12-month rose plot.

5.7. Period Planners

```
R> set.seed(123)
R> work <- sample(c("Study", "Adda", "Entertainment", "Games", "Exercise", "Meal"),
+   size = 24, replace = TRUE
+ )
R> plan_day(dwork = work, brdcol = NA)
```

Daily activity planner.

```
R> set.seed(10)
R> wtask <- c(
+   "Desk Work", "Field Work", "Visit", "Monitoring",
+   "Rest", "Reporting", "Meeting"
+ )
R> plan_week(wtask)
```

Weekly activity planner.

5.8. Case Study: US Accidents Analysis

This example demonstrates a full workflow: data import → cleaning → visualization → interpretation.

```
R> library(tidyverse)
R> library(clockplot)
R> # Import data (100-row sample from Kaggle US Accidents dataset)
R> acdt <- read.csv("https://raw.githubusercontent.com/mahmudstat/open-analysis/main/data/
R> # Basic clock chart - when do accidents happen?
R> p1 <- clock_chart(acdt, Time, Col = "darkred")
R> p1 + labs(title = "US Accidents: Time of Day")
R> # Multivariate: color by temperature, size by humidity
R> p2 <- clock_chart_qnt(acdt, time = Time, len = Humidity..., Col = Temperature.F.)
R> p2 + labs(size = "Humidity (%)", color = "Temperature (F)",
+   title = "US Accidents: Time, Temperature & Humidity")
R> # Facet by weather condition
R> p3 <- clock_chart_qlt(acdt, time = Time, crit = Weather_Condition) +
+   facet_wrap(~Weather_Condition) +
+   labs(color = "Weather", title = "Accidents by Weather Condition")
R> # Cyclic view: average humidity by hour (ensure 24 hours)
R> humidity_by_hour <- acdt %>%
+   mutate(hour = as.numeric(substr(Time, 1, 2))) %>%
+   group_by(hour) %>%
+   summarise(avg_humidity = mean(Humidity...), .groups = "drop") %>%
```



```
+   complete(hour = 0:23, fill = list(avg_humidity = NA)) %>%
+   pull(avg_humidity)
R> day_chart(hvalue = humidity_by_hour, high = "blue", low = "yellow") +
+   labs(title = "Average Humidity by Hour", fill = "Humidity (%)")
```

Interpretation

1. **Peak hours:** Accidents cluster around morning rush (6–9 AM) and evening (4–7 PM).
2. **Weather correlation:** Higher humidity (larger points) correlates with evening accidents.
3. **Weather conditions:** Accidents occur across various conditions (rain, overcast, mostly cloudy).
4. **Cyclic pattern:** Humidity peaks overnight (natural diurnal cycle), contributing to reduced visibility.

6. Summary and Discussion

clockplot provides a comprehensive toolkit for visualizing temporal event data on circular clock faces. Its key contributions are:

1. **Intuitive 24-hour clock visualization** that preserves circular continuity across midnight.
2. **Multivariate encoding** via color, size, and hand length on the same clock face.
3. **Generalized cyclic charts** for day/week/month/year periods using polar bar charts.
4. **Period planners** for visual scheduling of daily and weekly activities.
5. **Full ggplot2 compatibility** enabling faceting, theming, and customization.

The package is available on CRAN and GitHub (<https://github.com/mahmudstat/clockplot>). Future development may include jitter/stacking for simultaneous events, clock face customization (12-hour labels, adjustable start hour), and lollipop-style cyclic charts.

Computational Details

The results in this paper were obtained using R 4.6.1 with the **clockplot** 0.8.3 package, **ggplot2** 4.0.3, **dplyr** 1.2.1, and **tidyr** 1.3.2. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at <https://CRAN.R-project.org/>.

Acknowledgments

The example analyses use data from public repositories including the US Accidents Dataset (Kaggle) and earthquake data from the United States Geological Survey (USGS).

We are grateful to the Comprehensive R Archive Network (CRAN) (R Core Team 2025) team for maintaining the infrastructure that supports open source software distribution and reproducibility in R. We thank the tidyverse (Wickham, Averick, Bryan, Chang, McGowan, François, Golemund, Hayes, Henry, Hester, Kuhn, Pedersen, Miller, Bache, Müller, Ooms, Robinson, Seidel, Spinu, Takahashi, Vaughan, Wilke, Woo, and Yutani 2019) ecosystem for providing essential data manipulation and visualization functionalities. We also acknowledge GitHub for providing the collaborative platform used to develop, maintain, and share the source code for this package.

References

- Agostinelli C, Lund U (2025). *R package circular: Circular Statistics (version 0.5-2)*. URL <https://CRAN.R-project.org/package=circular>.
- Carvalho-Moreira JP, de Oliveira Guarnieri L, Passos MC, Emrich F, Bargi-Souza P, Peliciari-Garcia RA, Moraes MFD (2024). “CircadiPy: An open-source toolkit for analyzing chronobiology time series.” *Journal of Neuroscience Methods*, **411**, 110245. ISSN 0165-0270. doi: [10.1016/j.jneumeth.2024.110245](https://doi.org/10.1016/j.jneumeth.2024.110245). URL <https://www.sciencedirect.com/science/article/pii/S0165027024001900>.
- Golemund G, Wickham H (2011). “Dates and Times Made Easy with lubridate.” *Journal of Statistical Software*, **40**(3), 1–25. URL <https://www.jstatsoft.org/v40/i03/>.
- Lemon J (2006). “Plotrix: a package in the red light district of R.” *R-News*, **6**(4), 8–12.
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.
- Wickham H (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>.
- Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R, Golemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E, Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi K, Vaughan D, Wilke C, Woo K, Yutani H (2019). “Welcome to the tidyverse.” *Journal of Open Source Software*, **4**(43), 1686. doi: [10.21105/joss.01686](https://doi.org/10.21105/joss.01686).

A. Function Reference

Function	Data Arg	Time/Period Arg	Value/Metric Args	Key Options
<code>clock_chart()</code>	<code>data</code>	<code>time</code>	—	<code>Col</code> (single color)
<code>clock_chart_col()</code>	<code>data</code>	<code>time</code>	<code>crit</code> (numeric)	<code>high</code> , <code>low</code> (gradient)
<code>clock_chart_qnt()</code>	<code>data</code>	<code>time</code>	<code>len</code> , <code>Col</code> (numeric)	<code>high</code> , <code>low</code> (gradient)
<code>clock_chart_qlt()</code>	<code>data</code>	<code>time</code>	<code>crit</code> (factor)	Auto (Brewer Set2)
<code>cyclic_chart()</code>	<code>df</code>	<code>Period</code>	<code>Value</code> (numeric)	<code>ColV</code> , <code>crit</code>
<code>day_chart()</code>	—	— (24h implicit)	<code>hvalue</code> (len=24)	<code>high</code> , <code>low</code> , <code>width</code>
<code>week_chart()</code>	—	— (7d implicit)	<code>wvalue</code> (len=7)	<code>lgnm</code> , <code>high</code> , <code>low</code> , <code>width</code>
<code>year_chart()</code>	—	— (12m implicit)	<code>mvalue</code> (len=12)	<code>lgnm</code> , <code>high</code> , <code>low</code> , <code>width</code>
<code>plan_day()</code>	—	— (24h implicit)	<code>dwork</code> (char, len=24)	<code>width</code> , <code>brdcol</code>
<code>plan_week()</code>	—	— (7d implicit)	<code>wtask</code> (factor, len=7)	—

Table 4: Complete function reference for **clockplot**. All functions return ‘`ggplot`’ objects.

Affiliation:

Abdullah Al Mahmud

Bangladesh Cadet College

E-mail: mahmudstat@gmail.com

URL: <https://github.com/mahmudstat/clockplot>